- `create` returns the Active Record object regardless of whether it was successfully saved. You'll need to check the object for validation errors if you want to determine whether the data was written.

- `create!` returns the Active Record object on success; it raises an exception otherwise.

Let's look at this in a bit more detail.

Plain old `save` returns `true` if the model object is valid and can be saved:

```
if order.save
  # all OK
else
  # validation failed
end
```

It's up to us to check on each call to `save` to see that it did what we expected. The reason Active Record is so lenient is that it assumes `save` is called in the context of a controller's action method and the view code will be presenting any errors back to the end user. And for many applications, that's the case.

But if we need to save a model object in a context where we want to make sure to handle all errors programmatically, we should use `save!`. This method raises a `RecordInvalid` exception if the object could not be saved:

```
begin
  order.save!
rescue RecordInvalid => error
  # validation failed
end
```

Deleting Rows

Active Record supports two styles of row deletion. First, it has two class-level methods, `delete` and `delete_all`, that operate at the database level. The `delete` method takes a single ID or an array of IDs and deletes the corresponding row(s) in the underlying table. `delete_all` deletes rows matching a given condition (or all